

EXPERIMENT No. 9

Aim:

To understand Docker Architecture and Container Life Cycle, install Docker and execute docker commands to manage images and interact with containers.

Theory:

Docker is a popular platform that enables developers to build, package, and deploy applications as lightweight, portable, and self-sufficient containers. These containers encapsulate all the necessary dependencies and libraries required for an application to run, ensuring consistency across different environments. Here is a theoretical overview of Docker:

Containerization:

Docker utilizes containerization technology to create isolated environments for applications. Containers are lightweight, standalone, and executable packages that include everything needed to run an application, such as code, runtime, system tools, libraries, and settings.

This isolation ensures that applications run consistently across different environments, from development to production.

Docker Engine:

At the core of Docker is the Docker Engine, which is responsible for building, running, and managing containers. It consists of the Docker daemon, which manages containers, images, networks, and volumes, and the Docker client, which allows users to interact with the daemon through the Docker API.

Docker Images:

Docker images are read-only templates used to create containers. They contain the application code, runtime, libraries, dependencies, and other files needed to run the application. Images are built using Dockerfiles, which are text files that define the steps needed to create the image.

Docker Containers:

Containers are instances of Docker images that are running as isolated processes on a host machine. They are lightweight, portable, and can be easily started, stopped, moved, and deleted. Containers provide a consistent environment for applications to run, regardless of the underlying infrastructure.

Benefits of Docker:

Portability: Docker containers can run on any platform that supports Docker, making it easy to deploy applications across different environments.

Efficiency: Containers share the host OS kernel, reducing overhead and improving resource utilization.

Isolation: Containers provide a level of isolation that helps prevent conflicts between applications and dependencies.

Scalability: Docker enables easy scaling of applications by quickly spinning up additional containers.

Consistency: Docker ensures that applications run the same way in development, testing, and production environments.

Output:

The image displays two screenshots of the AWS Management Console, specifically the 'Create database' wizard for MySQL on Amazon RDS.

Top Screenshot: 'Choose a database creation method'

- Standard create** (selected): You set all of the configuration options, including ones for availability, security, backups, and maintenance.
- Easy create**: Use recommended best-practice configurations. Some configuration options can be changed after the database is created.

Engine options

- Engine type**:
 - Aurora (MySQL Compatible)**
 - Aurora (PostgreSQL Compatible)**
 - MySQL** (selected)
 - PostgreSQL**
 - MariaDB**
 - Oracle**

MySQL

MySQL is the most popular open source database in the world. MySQL on RDS offers the rich features of the MySQL community edition with the flexibility to easily scale compute resources or storage capacity for your database.

- Supports database size up to 64 TiB.
- Supports General Purpose, Memory Optimized, and Burstable Performance instance classes.
- Supports automated backup and point-in-time recovery.
- Supports up to 15 Read Replicas per instance, within a single Region or 5 read replicas cross-region.

Bottom Screenshot: 'Edition' and 'Engine version' sections

Edition

- MySQL Community** (selected)

Engine version

View the engine versions that support the following database features.

Hide filters

- Show only versions that support the Multi-AZ DB cluster** (selected): Create a Multi-AZ DB cluster with one primary DB instance and two readable standby DB instances. Multi-AZ DB clusters provide up to 2x faster transaction commit latency and automatic failover in typically under 35 seconds.
- Show only versions that support the Amazon RDS Optimized Writes**: Amazon RDS Optimized Writes improves write throughput by up to 2x at no additional cost.

Engine version

MySQL 8.0.40

Enable RDS Extended Support: Amazon RDS Extended Support is a paid offering. By selecting this option, you consent to being charged for this offering if you are running your database major version past the RDS end of standard support date for that version. Check the end of standard support date for your major version in the [RDS for MySQL documentation](#).

Templates

Choose a sample template to meet your use case.

- Production**: Use defaults for high availability and fast, consistent performance.
- Dev/Test**: This instance is intended for development use outside of a production environment.
- Free tier** (selected): Use RDS Free Tier to develop new applications, test existing applications, or gain hands-on experience with

Settings

DB instance identifier [info](#)
Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in the current AWS Region.
t1224
The DB instance identifier is case-insensitive, but is stored as all lowercase (as in "mydbinstance"). Constraints: 1 to 63 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

▼ Credentials Settings

Master username [info](#)
Type a login ID for the master user of your DB instance.
admin
1 to 16 alphanumeric characters. The first character must be a letter.

Credentials management
You can use AWS Secrets Manager or manage your master user credentials.

☐ **Managed in AWS Secrets Manager - most secure**
RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.

☒ **Self managed**
Create your own password or have RDS create a password that you manage.

☐ **Auto generate password**
Amazon RDS can generate a password for you, or you can specify your own password.

Master password [info](#)
[Masked password]
[Password strength](#) [Strong](#)

MySQL [>](#)

MySQL is the most popular open source database in the world. MySQL on RDS offers the rich features of the MySQL community edition with the flexibility to easily scale compute resources or storage capacity for your database.

- Supports database size up to 64 TiB.
- Supports General Purpose, Memory Optimized, and Burstable Performance instance classes.
- Supports automated backup and point-in-time recovery.
- Supports up to 15 Read Replicas per instance, within a single Region or 5 read replicas cross-region.

Instance configuration

The DB instance configuration options below are limited to those supported by the engine that you selected above.

DB instance class [info](#)

▼ Hide filters

☒ **Show instance classes that support Amazon RDS Optimized Writes** [info](#)
Amazon RDS Optimized Writes improves write throughput by up to 2x at no additional cost.

☒ **Include previous generation classes**

☐ Standard classes (includes m classes)

☐ Memory optimized classes (includes r and x classes)

☒ **Burstable classes (includes t classes)**

db.t4g.micro
2 vCPUs 1 GB RAM Network: Up to 2,048 Mbps

Storage

Storage type [info](#)
Provisioned IOPS SSD (io2) storage volumes are now available.

General Purpose SSD (gp2)
Baseline performance determined by volume size

Allocated storage [info](#)
20 GB

MySQL [>](#)

MySQL is the most popular open source database in the world. MySQL on RDS offers the rich features of the MySQL community edition with the flexibility to easily scale compute resources or storage capacity for your database.

- Supports database size up to 64 TiB.
- Supports General Purpose, Memory Optimized, and Burstable Performance instance classes.
- Supports automated backup and point-in-time recovery.
- Supports up to 15 Read Replicas per instance, within a single Region or 5 read replicas cross-region.

The screenshot shows the 'Create database' page in the AWS Management Console. The page is for creating a MySQL database. It includes sections for IAM role, Additional configuration, and Estimated monthly costs. The IAM role section shows the 'RDS service-linked role'. The Additional configuration section shows database options, encryption turned on, backup turned on, backtracking turned off, maintenance, CloudWatch Logs, and delete protection turned off. The Estimated monthly costs section lists the Amazon RDS Free Tier benefits: 750 hrs of Amazon RDS in a Single-AZ db.t2.micro, db.t3.micro or db.t4g.micro Instance, 20 GB of General Purpose Storage (SSD), and 20 GB for automated backup storage and any user-initiated DB Snapshots. A 'Create database' button is visible at the bottom right.

IAM role
The following service-linked role is used for publishing logs to CloudWatch Logs.
RDS service-linked role

Additional configuration
Database options, encryption turned on, backup turned on, backtracking turned off, maintenance, CloudWatch Logs, delete protection turned off.

Estimated monthly costs
The Amazon RDS Free Tier is available to you for 12 months. Each calendar month, the free tier will allow you to use the Amazon RDS resources listed below for free:

- 750 hrs of Amazon RDS in a Single-AZ db.t2.micro, db.t3.micro or db.t4g.micro Instance.
- 20 GB of General Purpose Storage (SSD).
- 20 GB for automated backup storage and any user-initiated DB Snapshots.

[Learn more about AWS Free Tier](#)
When your free usage expires or if your application use exceeds the free usage tiers, you simply pay standard, pay-as-you-go service rates as described in the [Amazon RDS Pricing page](#).

You are responsible for ensuring that you have all of the necessary rights for any third-party products or services that you use with AWS services.

[Cancel](#) [Create database](#)

The screenshot shows the 'Instances' page in the AWS Management Console. It displays a table of EC2 instances. The table has columns for Name, Instance ID, Instance state, Instance type, Status check, Alarm status, Availability Zone, and Public IP. One instance is listed: t1224, i-09c449653e518cac4, Running, t2.micro, Initializing, View alarms, ap-south-1a, ec2-3-108. Below the table, the details for instance i-09c449653e518cac4 (t1224) are shown. The details include the Instance ID, Instance type, Public IPv4 address (3.108.67.99), Private IPv4 addresses (172.30.0.203), and Public IPv4 DNS (ec2-3-108-67-99.ap-south-1.compute.amazonaws.com).

Instances (1/1)

Find instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IP
t1224	i-09c449653e518cac4	Running	t2.micro	Initializing	View alarms	ap-south-1a	ec2-3-108

i-09c449653e518cac4 (t1224)

Instance summary

Instance ID: i-09c449653e518cac4

Instance type: t2.micro

Public IPv4 address: 3.108.67.99 | [open address](#)

Private IPv4 addresses: 172.30.0.203

Public IPv4 DNS: ec2-3-108-67-99.ap-south-1.compute.amazonaws.com

```
aws [Alt+S]
[ec2-user@ip-172-30-0-203 ~]$ sudo yum install -y docker
Amazon Linux 2023 Kernel livepatch repository 115 KB/s | 15 KB 00:00
Dependencies resolved.
-----
Package Architecture Version Repository Size
-----
Installing:
docker x86_64 25.0.6-1.amzn2023.0.1 amazonlinux 44
Installing dependencies:
containerd x86_64 1.7.25-1.amzn2023.0.1 amazonlinux 36
iptables-libse x86_64 1.6.0-3.amzn2023.0.2 amazonlinux 401
iptables-nft x86_64 1.6.0-3.amzn2023.0.2 amazonlinux 183
libnftables x86_64 3.0-1.amzn2023.0.1 amazonlinux 75
libnftfilter_comtrack x86_64 1.0.0-2.amzn2023.0.2 amazonlinux 50
libnftnl x86_64 1.0.1-19.amzn2023.0.2 amazonlinux 30
libnftnl x86_64 1.2.2-2.amzn2023.0.2 amazonlinux 84
pigz x86_64 2.5-1.amzn2023.0.3 amazonlinux 83
runc x86_64 1.2.4-1.amzn2023.0.1 amazonlinux 3.4
Transaction Summary
-----
Install 10 Packages
Total download size: 84 M
Installed size: 319 M
Downloading Packages:
(1/10): iptables-libse-1.6.0-3.amzn2023.0.2.x86_64.rpm 7.3 MB/s | 401 KB 00:00
(2/10): iptables-nft-1.6.0-3.amzn2023.0.2.x86_64.rpm 7.0 MB/s | 183 KB 00:00
(3/10): libnftables-3.0-1.amzn2023.0.1.x86_64.rpm 3.4 MB/s | 75 KB 00:00
(4/10): libnftfilter_comtrack-1.0.0-2.amzn2023.0.2.x86_64.rpm 1.3 MB/s | 50 KB 00:00
(5/10): libnftnl-1.0.1-19.amzn2023.0.2.x86_64.rpm 1.3 MB/s | 30 KB 00:00
(6/10): libnftnl-1.2.2-2.amzn2023.0.2.x86_64.rpm 3.8 MB/s | 84 KB 00:00
(7/10): pigz-2.5-1.amzn2023.0.3.x86_64.rpm 2.7 MB/s | 83 KB 00:00
(8/10): runc-1.2.4-1.amzn2023.0.1.x86_64.rpm 24 MB/s | 3.4 MB 00:00
(9/10): containerd-1.7.25-1.amzn2023.0.1.x86_64.rpm 44 MB/s | 36 MB 00:00
(10/10): docker-25.0.6-1.amzn2023.0.1.x86_64.rpm 40 MB/s | 44 MB 00:01
-----
Total 74 MB/s | 84 MB 00:01
-----
Transaction Summary
-----
Install 10 Packages
Total download size: 84 M
Installed size: 319 M
Downloading Packages:
(1/10): iptables-libse-1.6.0-3.amzn2023.0.2.x86_64.rpm 7.3 MB/s | 401 KB 00:00
(2/10): iptables-nft-1.6.0-3.amzn2023.0.2.x86_64.rpm 7.0 MB/s | 183 KB 00:00
(3/10): libnftables-3.0-1.amzn2023.0.1.x86_64.rpm 3.4 MB/s | 75 KB 00:00
(4/10): libnftfilter_comtrack-1.0.0-2.amzn2023.0.2.x86_64.rpm 1.3 MB/s | 50 KB 00:00
(5/10): libnftnl-1.0.1-19.amzn2023.0.2.x86_64.rpm 1.3 MB/s | 30 KB 00:00
(6/10): libnftnl-1.2.2-2.amzn2023.0.2.x86_64.rpm 3.8 MB/s | 84 KB 00:00
(7/10): pigz-2.5-1.amzn2023.0.3.x86_64.rpm 2.7 MB/s | 83 KB 00:00
(8/10): runc-1.2.4-1.amzn2023.0.1.x86_64.rpm 24 MB/s | 3.4 MB 00:00
(9/10): containerd-1.7.25-1.amzn2023.0.1.x86_64.rpm 44 MB/s | 36 MB 00:00
(10/10): docker-25.0.6-1.amzn2023.0.1.x86_64.rpm 40 MB/s | 44 MB 00:01
-----
Total 74 MB/s | 84 MB 00:01
-----
```

```

AWS [Alt+S]
Search
Verifying : pigz-2.5-1.amzn2023.0.3.x86_64 9/1
Verifying : runc-1.2.4-1.amzn2023.0.1.x86_64 10/1

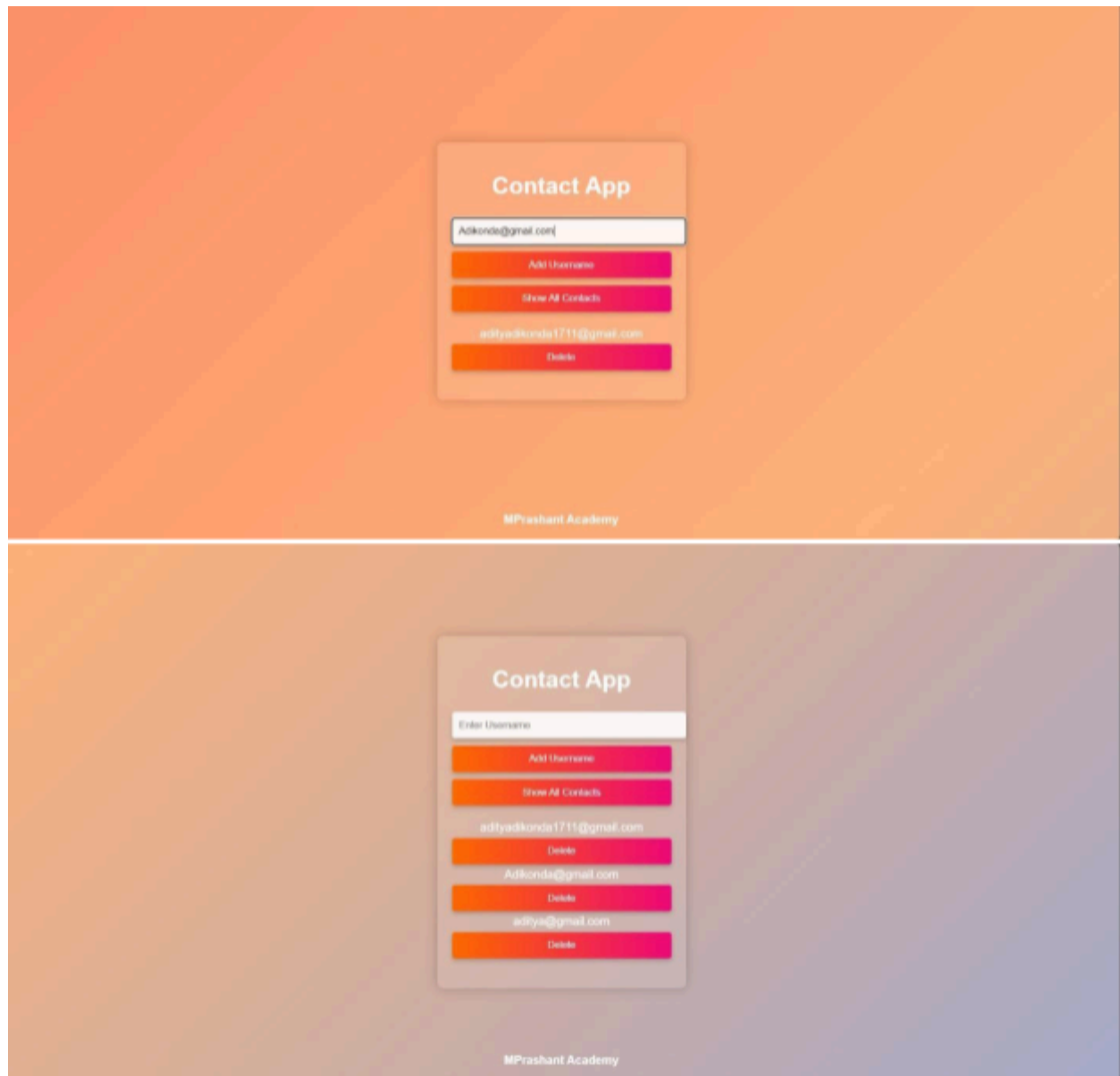
Installed:
containerd-1.7.25-1.amzn2023.0.1.x86_64      docker-25.0.8-1.amzn2023.0.1.x86_64      iptables-libs-1.8.6-3.amzn2023.0.2.x86_64
iptables-soft-1.8.6-3.amzn2023.0.2.x86_64      libbgroup-3.0-1.amzn2023.0.1.x86_64      libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64
libnftnl-1.0.1-19.amzn2023.0.2.x86_64      libnftal-1.2.2-2.amzn2023.0.2.x86_64      pigz-2.5-1.amzn2023.0.3.x86_64
runc-1.2.4-1.amzn2023.0.1.x86_64

Complete!
ec2-user@ip-172-30-0-203 ~$ sudo systemctl start docker
ec2-user@ip-172-30-0-203 ~$ sudo systemctl status docker
* docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/systemd/docker.service; disabled; preset: disabled)
   Active: active (running) since Tue 2025-04-01 13:34:08 UTC; 11s ago
   TriggeredBy: * docker.socket
   Docs: https://docs.docker.com
   Process: 27033 ExecStartPre=/bin/sh mkdir -p /run/docker (code=exited, status=0/SUCCESS)
   Process: 27034 ExecStartPre=/usr/libexec/docker/docker-setup-runtimes.sh (code=exited, status=0/SUCCESS)
   Main PID: 27035 (dockerd)
   Tasks: 7
   Memory: 28.1M
   CPU: 250ms
   CGroup: /system.slice/docker.service
           └─27035 /usr/bin/dockerd -B fd:/// --containerd=/run/containerd/containerd.sock --default-ulimit nofile=32768:65536

Apr 01 13:34:07 ip-172-30-0-203.ap-south-1.compute.internal systemd[1]: Starting docker.service - Docker Application Container Engine...
Apr 01 13:34:07 ip-172-30-0-203.ap-south-1.compute.internal dockerd[27035]: time="2025-04-01T13:34:07.656215534Z" level=info msg="Starting up"
Apr 01 13:34:07 ip-172-30-0-203.ap-south-1.compute.internal dockerd[27035]: time="2025-04-01T13:34:07.715129546Z" level=info msg="Loading containers: start."
Apr 01 13:34:08 ip-172-30-0-203.ap-south-1.compute.internal dockerd[27035]: time="2025-04-01T13:34:08.117426754Z" level=info msg="Loading containers: done."
Apr 01 13:34:08 ip-172-30-0-203.ap-south-1.compute.internal dockerd[27035]: time="2025-04-01T13:34:08.141367092Z" level=info msg="Daemon has completed initialization"
Apr 01 13:34:08 ip-172-30-0-203.ap-south-1.compute.internal dockerd[27035]: time="2025-04-01T13:34:08.182701850Z" level=info msg="API listen on /run/docker.sock"
Apr 01 13:34:08 ip-172-30-0-203.ap-south-1.compute.internal systemd[1]: Started docker.service - Docker Application Container Engine.

lines 1-22/22 (0 lines)
ec2-user@ip-172-30-0-203 ~$ sudo docker pull philippaul/node-mysql-app:02
02: Pulling from philippaul/node-mysql-app
ff1d7c41c74: Pull complete
8253aea4ea7: Pull complete
bd2201bd995c: Pull complete
de76e268b10: Pull complete
f9a8df589451: Pull complete
c751ee005dea: Pull complete
cf32e33c3f27: Pull complete
0cf0c2f24a4d: Pull complete
dd7a3b6d132: Pull complete
c30ca8a95db0: Pull complete
6a182df3db1: Pull complete
f561a7b8e97: Pull complete
ff7978b844b1: Pull complete
Digest: sha256:f7c1cfdb42a2f4a406426b093f8b83bac6ef3f98d0682c943f395bf9e42966b
Status: Downloaded newer image for philippaul/node-mysql-app:02
docker.io/philippaul/node-mysql-app:02
ec2-user@ip-172-30-0-203 ~$

```



```
batch the logs of a container
ec2-user@ip-172-30-0-203 ~$ sudo docker run -it --rm mysql:8.0 mysql -h ti224.c3aaqis4wqsg-ap-south-1-rds.amazonaws.com -u admin -p
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
e6a72a6e83b: Pull complete
8e01aa53f13: Pull complete
5fa3211d7a7: Pull complete
73306441f7a6: Pull complete
d339a14f4a1: Pull complete
a386f934eb3: Pull complete
3272c957f26: Pull complete
106a802288: Pull complete
16f4325df2d: Pull complete
d34979c712d: Pull complete
e67a2f637eb: Pull complete
Digest: sha256:b5f577825b52ab201d6281fb281aabbfc73507eda8f2c274579025153ef0306
Status: Downloaded newer image for mysql:8.0
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 20
Server version: 8.0.40 Source distribution

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

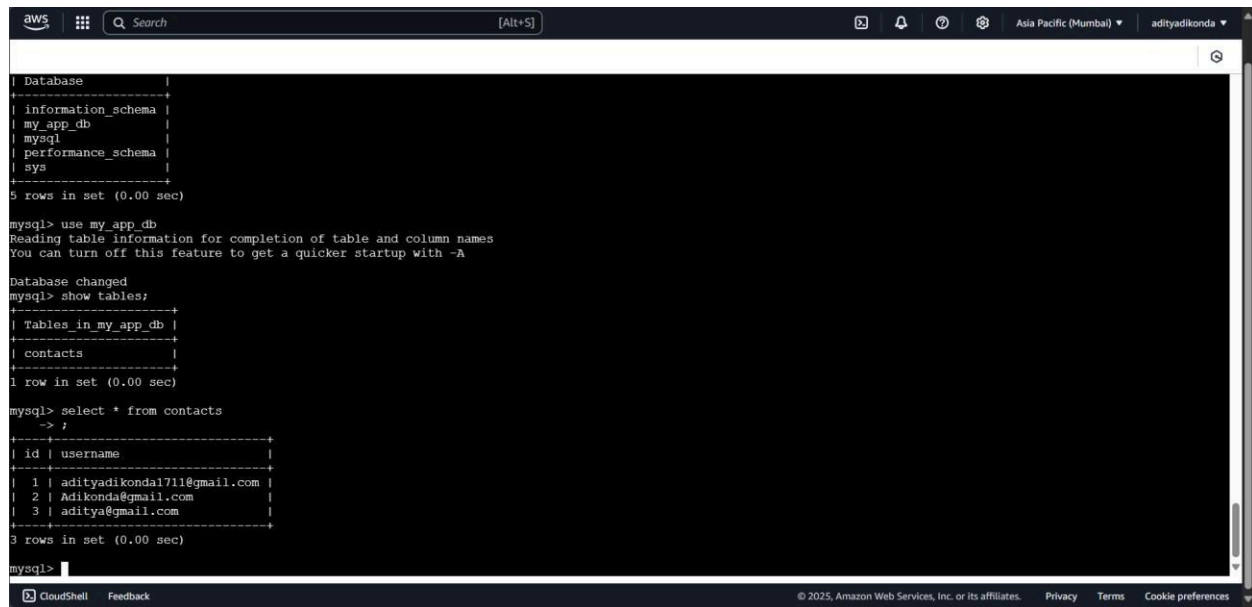
mysql> show databases
+-----+
| Database |
+-----+
mysql>

mysql> show databases
+-----+
| Database |
+-----+
| information_schema |
| my_app_db |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.00 sec)

mysql> use my_app_db
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
+-----+
| Tables_in_my_app_db |
+-----+
| contacts |
+-----+
1 row in set (0.00 sec)

mysql> select * from contacts
+-----+
| id | username |
+-----+
| 1 | adityadikondal71@gmail.com |
| 2 | Aditondal@gmail.com |
+-----+
```



The screenshot shows an AWS CloudShell terminal window with a dark background. The terminal displays the output of several MySQL commands. At the top, the AWS logo and a search bar are visible. The terminal content includes a list of databases, a command to use a specific database, a command to show tables, and a query to retrieve data from a table. The output is formatted with ASCII art tables.

```
aws
Search [Alt+S]
Asia Pacific (Mumbai) adityadikonda

Database
+-----+
| information_schema |
| my_app_db          |
| mysql              |
| performance_schema |
| sys                |
+-----+
5 rows in set (0.00 sec)

mysql> use my_app_db
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
+-----+
| Tables in my_app_db |
+-----+
| contacts             |
+-----+
1 row in set (0.00 sec)

mysql> select * from contacts
->
+-----+
| id | username                               |
+-----+
| 1  | adityadikondal711@gmail.com          |
| 2  | Adikonda@gmail.com                  |
| 3  | aditya@gmail.com                    |
+-----+
3 rows in set (0.00 sec)

mysql>
```

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Conclusion:

Docker revolutionizes the software development and deployment process by providing a powerful platform for containerization. By encapsulating applications and their dependencies into lightweight, portable containers,